

ENTRY-LEVEL APPLICATION

하드웨어의 동작을 이해하고, 소프트웨어로 연결합니다.

전지오

Jioh Jeon

임베디드 · 시스템 소프트웨어

IPG Automotive Korea 재직 중 · 신입 지원

- 01 **Hardware control**
STM32 · GPIO · UART · TIM1 / DMA
- 02 **System software**
C / C++ · Linux · Threads · TCP
- 03 **System integration**
Interfaces · Lifecycle · Validation

- STM32 주변장치 제어부터 Linux 장치 모듈까지의 구현 경험
- 모델·통신·실장비 사이의 문제를 분석하고 연결한 실무 경험
- 프로젝트별 본인 기여, 설계 판단, 결과와 확인 범위 정리

wldh0026@gmail.comgithub.com/gitgio99

프로젝트 한눈에 보기

관심 기술에서 출발해, 역할과 구현 근거까지 확인할 수 있는 8개 프로젝트

01	STM32 디스플레이 제어와 DMA 확장	03
	주변장치 제어 · 데이터 이동 · 펌웨어 구조	
02	Linux TCP 원격 장치 제어	05
	Linux 시스템 프로그래밍 · 장치 추상화 · 네트워크	
03	실시간 차량 모델·시험장비 연동	07
	요구사항 해석 · 인터페이스 정합성 · 체계 통합	
04	GPU 이미지·라벨 생성 파이프라인	09
	Linux / C++ · 데이터 파이프라인 · 동기화	
05	실시간 타깃 빌드 환경 업그레이드	11
	빌드 의존성 · 타깃 이해 · 시스템 디버깅	
06	LiDAR·카메라 데이터 정합과 변형	12
	센서 인터페이스 · 좌표 정합 · 데이터 처리	
07	CCTV 유지보수 예약 관리	13
	C++ 모듈 설계 · 요구사항 구현 · 팀 협업	
08	카메라 기반 AI 모빌리티	14
	HW/SW 통합 · 센서 처리 · 제어 흐름	

읽는 순서 요약과 역할 → 처리 흐름 → 본인 기여 → 문제 해결 → 성장과 결과

개발 경험과 교육 이수 항목을 구분하고, 팀 성과에는 팀 기여 범위 표시

01 STM32 디스플레이 제어와 DMA 확장

이면도로 사각지대 사고방지 시스템 · VEDA 팀 프로젝트 / 후속 확장

2025.06-08 · DMA 확장 2025.09 코드 기준 | STM32 하드웨어 제어 및 통신 구현

경고 명령을 실제 디스플레이 출력으로 연결하고, 반복 GPIO 제어를 타이머·DMA 전송 구조로 확장

C / STM32F401RE / UART / CRC16 / TIM1 / DMA / GPIO BSRR

PROCESS FLOW



CPU: 제어 워드 생성 · DMA: GPIO 전송 · 콜백: 버퍼 갱신 시점 관리

01 본인 기여

- **UART 프레임:** STX/ETX/DLE 기반 파싱, CRC16 검증 및 ACK/NACK 응답 구현
- **출력 제어:** 경고 표시·화면 ON/OFF-RTC 시각 동기화 구현
- **후속 확장:** TIM1 이벤트와 DMA 멀티버퍼를 이용한 BSRR 전송 구조 구현

02 설계 판단과 문제 해결

- **핀 배치 제약:** 분산된 신호를 GPIOB로 통합하여 단일 BSRR 목적지에 맞춘 핀맵 재구성
- **연산과 전송 분리:** 픽셀을 32-bit 제어 워드로 사전 변환하고 RAM 사용량과 CPU 작업량의 교환관계 검토
- **버퍼 동기화:** DMA 완료 콜백과 유휴 플래그를 이용해 출력 버퍼와 갱신 버퍼 구분

03 성장과 학습

- 레지스터·타이머·DMA의 연결 관계 이해 및 인터럽트 기반 버퍼 관리 경험
- 프로토콜 수신 처리와 물리 출력 타이밍을 구분하는 펌웨어 설계 경험

04 결과와 확인 범위

- **팀 성과:** 한화비전 VEDA 최종 프로젝트 최우수상(1위), 2025.08.07
- **구현 결과:** GPIO / DMA 두 버전의 출력 구조 및 UART 진단 코드 구성

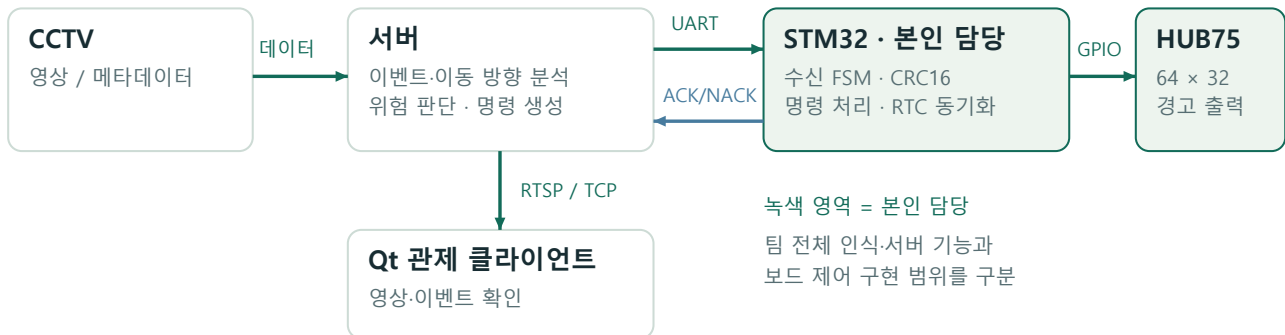
<https://github.com/gitgio99/stm-32>

시스템 구성도

STM32 디스플레이 제어와 DMA 확장

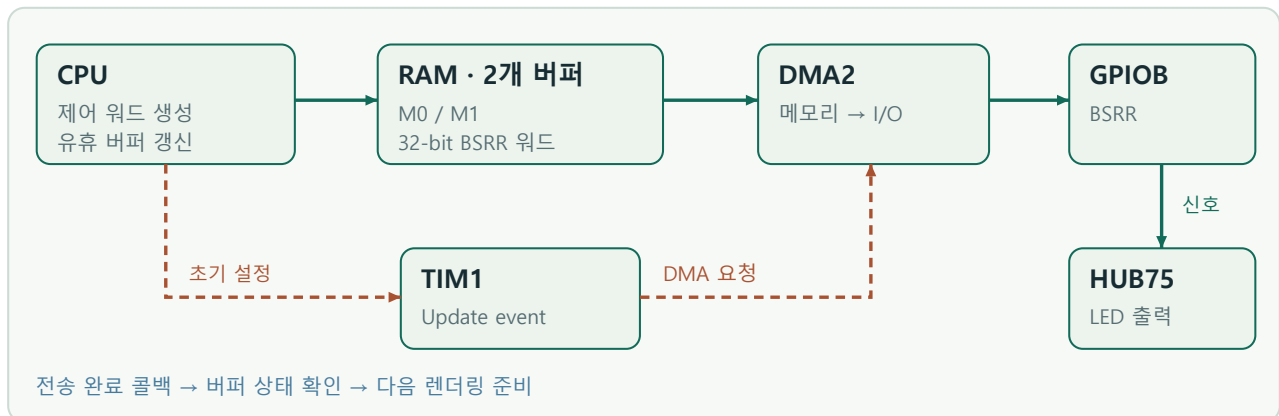
01 경고 이벤트에서 실제 출력까지

팀 시스템 속 본인 담당: STM32 통신-커맨드 처리·HUB75 제어



02 후속 확장: 타이머·DMA 기반 출력

CPU는 제어 워드를 준비하고, DMA는 GPIO 레지스터에 전송



설계 포인트: 명령 처리 / 출력 타이밍 / 데이터 전송을 분리

구성도에서 확인할 핵심 역량

- 팀 시스템: CCTV 메타데이터 → 서버 위험 판단 → STM32 경고 출력
- 본인 담당: UART 프로토콜·수신 FSM·CRC16·HUB75 제어
- 후속 확장: TIM1 트리거·DMA 전송·멀티버퍼 상태 관리

02 Linux TCP 원격 장치 제어

Raspberry Pi 서버와 Ubuntu 클라이언트 · VEDA 프로젝트

2025.05.21-23 | C 기반 클라이언트-서버 및 장치 모듈 개발

네트워크 명령을 장치별 공유 라이브러리로 분리해 LED·부저·센서·7세그먼트를 제어

C / Linux / TCP sockets / pthread / dlopen / dlsym

PROCESS FLOW

Ubuntu CLI



TCP / pthread



dlopen → .so



Raspberry Pi GPIO

사용자 공간 제어: TCP 명령 → 동적 심볼 조회 → wiringPi 장치 제어

01 본인 기여

- 통신 구현: TCP 클라이언트-서버와 클라이언트별 pthread 처리 구성
- 모듈 분리: LED·부저·조도센서·7세그먼트 제어를 .so 라이브러리로 구성
- 운영 기능: 데몬화, 로그 출력, Makefile 기반 빌드 구성

02 설계 판단과 문제 해결

- 장치별 분기: 명령에서 장치명을 해석하고 dlopen/dlsym으로 제어 함수 연결
- 긴 작업 분리: 부저 재생을 별도 스레드로 실행하는 구조 적용
- 실행 환경: 데몬의 작업 디렉터리 변화에 맞춰 라이브러리·로그 경로 명시

03 성장과 학습

- 소켓·프로세스·스레드·동적 링크가 결합되는 Linux 실행 구조 이해
- 명령 처리와 하드웨어 접근을 모듈 경계로 분리하는 인터페이스 설계 경험

04 결과와 확인 범위

- 구현 결과: 원격 CLI에서 4종 장치의 명령 처리 및 상태 응답 구성
- 개발 범위: wiringPi 기반 사용자 공간 장치 제어와 공유 라이브러리 연동

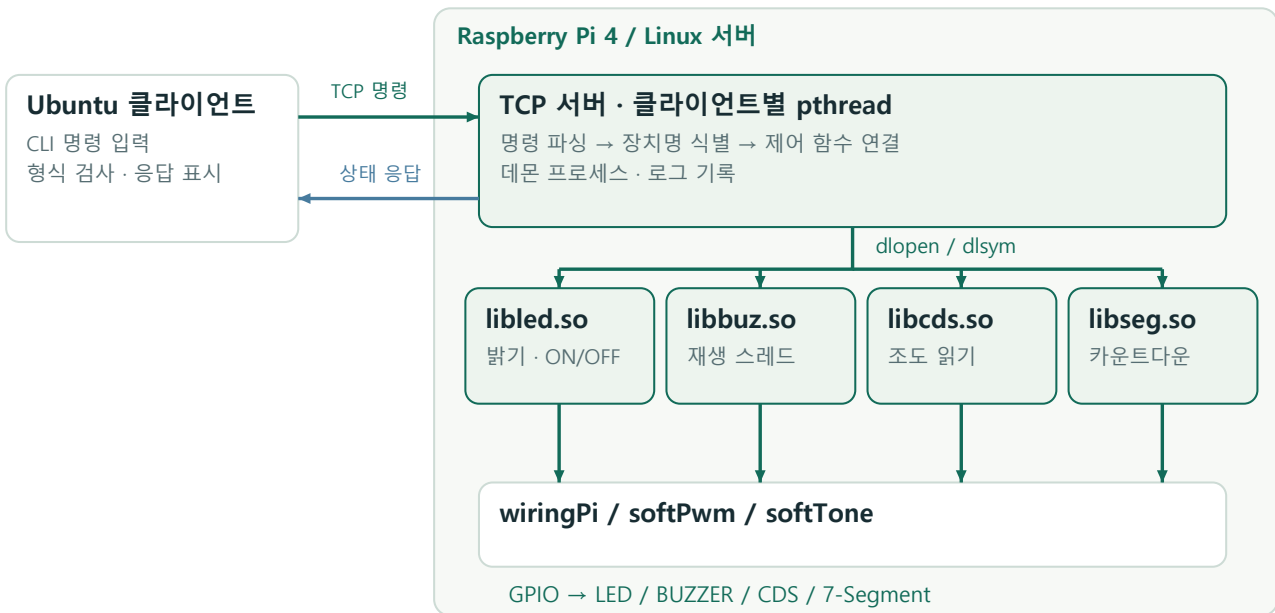
https://github.com/gitgio99/TCP_module_control

시스템 구성도

Linux TCP 원격 장치 제어

네트워크 명령을 장치 제어 모듈로 연결

사용자 공간 SW: 클라이언트 → 서버 → 공유 라이브러리 → GPIO



핵심 역량: 소켓-스레드 / 동적 링크 / 장치 모듈화

구성도에서 확인할 핵심 역량

- 통신 계층: TCP 요청과 응답, 클라이언트별 스레드 처리
- 모듈 계층: 장치별 공유 라이브러리와 동적 함수 연결
- 장치 계층: wiringPi 기반 GPIO 제어 및 부저 재생 스레드

03 실시간 차량 모델·시험장비 연동

차량 모델·시험장비 연동 · 실무 프로젝트

2026.03-06 · 후속 기술지원 | C 제어 로직·신호 매핑·통합 검증

차량 모델과 실장비 사이의 신호 의미·단위·상태를 맞추고, 시험 결과로 연동 문제를 추적

C / Real-time model / Control logic / CAN / RBS / System integration

PROCESS FLOW

Vehicle model → Signal mapping → CAN / RBS → Test equipment

명령 송신 → 실측 피드백 → 모델 반영 · 구성도는 역할 중심 요약

01 본인 기여

- 제어 로직: 토크 분배·변화량 제한·상태 초기화를 위한 C 로직 구현
- 인터페이스: 모델 변수와 시험장비 신호의 CAN/RBS 매핑 검토·수정
- 통합 검증: Office·실장비 환경의 주행 결과 비교 및 결과 자료 작성

02 설계 판단과 문제 해결

- 초기화 순서: 실행 시 생성되는 모델 변수의 연결 시점과 유효성 점검
- 신호 정합성: 토크·휠 속도·모터 속도의 의미와 변환 관계를 구분하여 매핑 검토
- 상태 전환: 시험 시작 시 이전 상태 초기화, 종료 시 출력 정리 로직 구성

03 성장과 학습

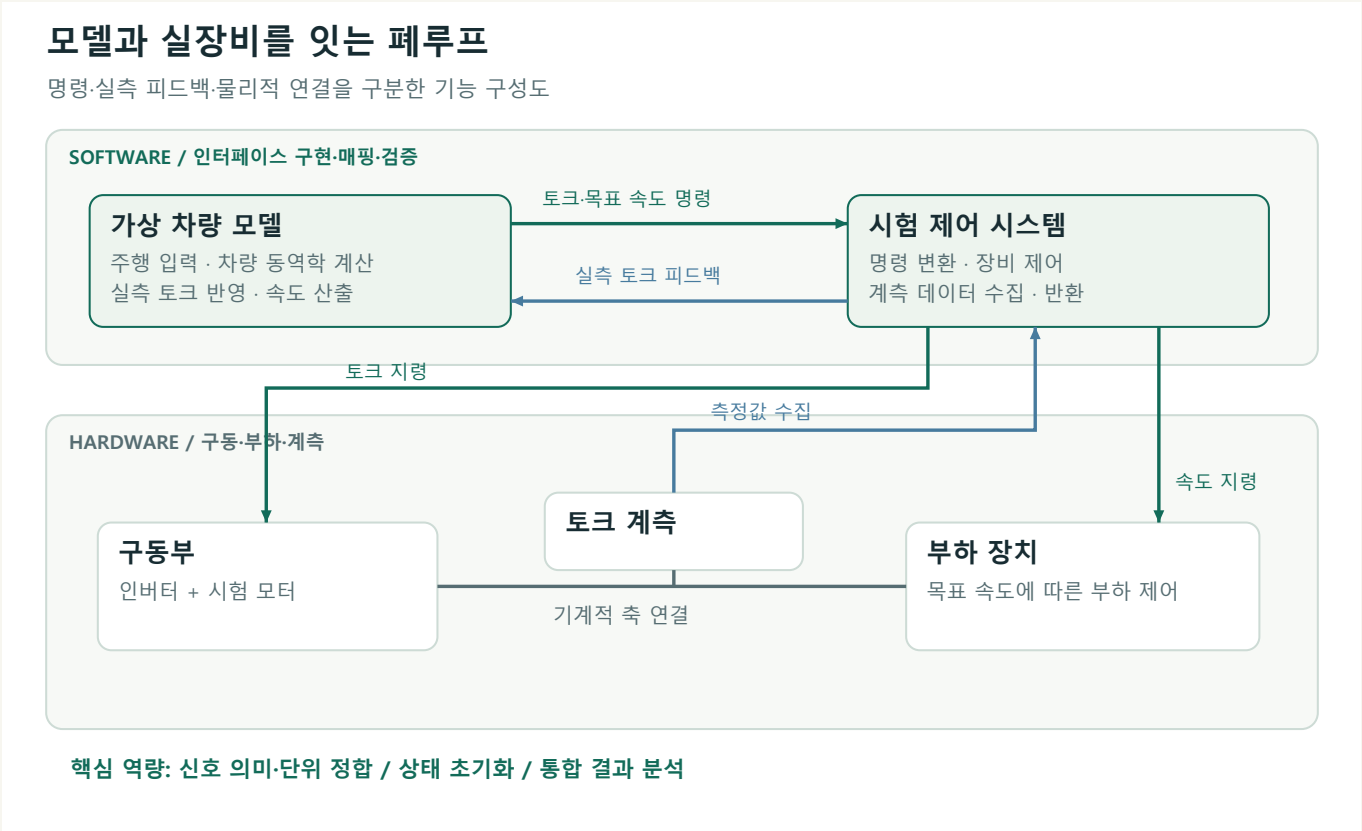
- 코드·모델·통신·물리 장비를 함께 보는 시스템 단위 원인 분석 경험
- 이상적 시뮬레이션과 실장비 응답 지연을 구분하는 시험 결과 해석

04 결과와 확인 범위

- 보고서 결과: 페루프 통신·매핑 동작 확인 및 주행 시나리오 비교 정리
- 관찰 사항: 실장비 응답 지연에 따른 초기 속도 차이를 결과에 명시

시스템 구성도

실시간 차량 모델·시험장비 연동



구성도에서 확인할 핵심 역량

- **명령 흐름:** 모델의 토크·목표 속도를 장비 제어 입력으로 연결
- **피드백 흐름:** 실제 토크 측정값을 모델 입력에 반영
- **본인 기여:** 신호 매핑·단위 정합·상태 초기화·통합 결과 분석

04 GPU 이미지·라벨 생성 파이프라인

기상 조건별 인식 평가를 위한 개인 프로젝트 · IPG Automotive Korea

2026 · 신입 개인 프로젝트 | C++/CUDA 이미지 추출·라벨 생성·동기화

GPU 이미지와 물리 센서 정보를 같은 프레임 기준으로 묶어 학습용 데이터 생성 과정을 자동화

C++17 / Linux / CUDA / nvJPEG / Python / Coordinate transforms

PROCESS FLOW

GPU RGB → JPEG



SimCore.Time ID



Image / Label pairs

GT → 2D labels



병렬 생성되는 이미지·라벨을 시뮬레이션 시간 기반 인덱스로 연결

01 본인 기여

- **이미지 추출:** RSDA GPU 플러그인과 nvJPEG를 활용한 캡처·저장 경로 구성
- **라벨 생성:** 3D 객체의 코너점을 카메라 좌표로 투영하고 YOLO 형식으로 변환
- **평가 구성:** 날씨별 데이터 생성과 YOLOv8m 학습·비교 흐름 구축

02 설계 판단과 문제 해결

- **시점 불일치:** 이미지 로깅과 물리 계산의 차이를 프레임 대응 문제로 정의
- **공통 인덱스:** 단순 카운터 대신 SimCore.Time 기반의 프레임 번호 계산 적용
- **좌표 정합성:** FOV·해상도·객체 회전을 반영한 투영과 정규화 처리

03 성장과 학습

- GPU 처리 경로와 CPU 메타데이터 처리의 연결 및 시간 동기화 경험
- 모델 성능 검토에 앞서 입력 데이터 정합성을 확인하는 문제 해결 경험

04 결과와 확인 범위

- **구현 결과:** 이미지 저장·GT 라벨 생성·프레임 인덱싱 자동화
- **평가 구성:** 기상 조건별 이미지·라벨을 활용한 모델 학습과 결과 비교

시스템 구성도

GPU 이미지·라벨 생성 파이프라인

별도로 생성되는 이미지와 라벨의 프레임 맞추기

물리 계산과 이미지 저장 시점의 차이를 시간 기반 인덱스로 보정

01 GPU 이미지 경로

카메라 RGB

GPU 센서 데이터
RSDA 플러그인 입력

nvJPEG 인코딩 · 저장

JPEG 이미지 생성
센서 cycleNumber 기반 이름

SimCore.Time 기반 라벨 인덱스

02 CPU 라벨 경로

Ground Truth 객체

3D 위치 · 크기 · 회전
물리 센서 메타데이터

3D → 2D 투영

카메라 FOV · 해상도 반영
YOLO 라벨 · 프레임 번호 생성

이미지·라벨 대응

Frame ID 기준 연결
저장 파일명 규칙 정렬
시간 오프셋 보정 반영

기상 조건별 학습·평가 데이터

동일 프레임의 JPEG 이미지와 YOLO 라벨을 묶어 데이터셋 구성
생성 데이터로 모델 학습 및 기상 조건별 결과 비교

핵심 역량: GPU/CPU 경로 연동 / 시간 기반 프레임 보정 / 좌표 변환

구성도에서 확인할 핵심 역량

- **이미지 경로:** GPU RGB를 nvJPEG로 인코딩하고 센서 cycleNumber로 파일명 생성
- **라벨 경로:** 3D 객체를 2D로 투영하고 SimCore.Time 기반 프레임 인덱스 적용
- **동기화 기준:** 물리 계산·이미지 저장 시점의 차이를 보정해 프레임 대응 정렬

05 실시간 타겟 빌드 환경 업그레이드

시뮬레이션·시험 플랫폼 기술지원 · IPG Automotive Korea

2026.06 · 현장 기술지원 | 빌드 설정·타겟 호환성·연동 환경 분석

호스트와 타겟의 차이를 기준으로 빌드 경로·라이브러리·런타임 환경의 호환성 문제를 분해

Linux target / Python build scripts / QNX / Linux / Static / shared libs / dSPACE

PROCESS FLOW

Host build setup → Target architecture → Libraries / firmware → Runtime / network

타겟별 설정과 의존성을 단계별로 검토한 업그레이드 지원 경험

01 본인 기여

- **설정 검토:** 버전별 빌드 스크립트·검색 경로·사용자 라이브러리 비교
- **호환성 분석:** QNX/Linux 타겟과 32/64비트 라이브러리 차이 검토
- **현장 지원:** 빌드 로그·펌웨어 버전·네트워크 설정의 점검 과정 기록

02 설계 판단과 문제 해결

- **경로 오류:** 다른 환경의 검색 경로가 남은 설정을 확인하고 수정
- **타겟 차이:** 호스트 OS가 아닌 실제 실행 타겟 기준으로 라이브러리 형식 검토
- **단계별 점검:** 모델 빌드·타겟 펌웨어·NFS 연결을 나눠 오류 발생 구간 추적

03 성장과 학습

- 임베디드 SW의 빌드 산출물과 실제 실행 환경 사이의 의존성 이해
- 로그와 버전 차이를 근거로 원인 후보를 좁히는 기술지원 경험

04 결과와 확인 범위

- **확인 결과:** 경로 수정 및 타겟·라이브러리 호환성 검토 내용 문서화
- **업무 산출물:** 버전별 의존성·설정 변경 사항을 기록하여 현장 점검에 활용

06 LiDAR·카메라 데이터 정합과 변형

센서 퓨전 이상 탐지 연구 지원 · 학부연구생

2024.03-12 | 센서 데이터 전처리·시각화·변형 데이터 생성

서로 다른 센서 좌표와 채널을 연결하고, 객체 단위 변형으로 이상 상황 분석용 데이터 구성

Python / KITTI / nuScenes / Open3D / NumPy / Calibration

PROCESS FLOW

LiDAR / Camera



Calibration



Projection / Align



5-channel data

센서 융합 데이터 구성과 객체 단위 복사·이동·삭제 실험

01 본인 기여

- 데이터 처리: KITTI·nuScenes 포인트 클라우드의 시각화 및 변형
- 융합 입력: 카메라 RGB와 LiDAR intensity-depth를 결합한 5채널 데이터 생성
- 실험 데이터: Bounding Box 단위의 포인트 복사·이동·삭제 처리 자동화

02 설계 판단과 문제 해결

- 좌표계 차이: 보정 정보를 적용해 3D 포인트와 이미지 픽셀 대응 확인
- 채널 정합: RGB·거리·반사 강도의 의미를 구분해 융합 입력 구성
- 반복 실험: 객체 단위 변형 절차를 코드화하여 수작업 반복 축소

03 성장과 학습

- 다중 센서 데이터의 좌표 변환·투영·전처리 흐름 이해
- 시각화를 통해 데이터 처리 결과를 검토하는 연구 작업 경험

04 결과와 확인 범위

- 산출물: 5채널 생성 및 객체 변형 실험 노트북 구성
- 결과 점검: 객체 단위 변형으로 이상 상황을 구성하고 융합 입력을 시각적으로 확인

07 CCTV 유지보수 예약 관리

C++ / Qt 기반 업무 도구 · VEDA 팀 프로젝트

2025 · VEDA 교육과정 | 예약 등록·조회 기능 담당

업무 규칙을 예약 검증 로직으로 옮기고, 일정 등록과 조회 화면을 연결

C++17 / Qt 6 / Signal / Slot / JSON / CMake

PROCESS FLOW

Register



Validate capacity



JSON data



Schedule view

본인 담당 범위: 예약 등록·조회 · 팀 공통 데이터 구조와 연동

01 본인 기여

- 등록 화면: 날짜·시간 선택 및 예약 검증 로직 구현
- 조회 화면: 날짜별 일정 표시 및 예약 수정·삭제 흐름 구현

02 설계 판단과 문제 해결

- 업무 제약: 시간대별 예약 수를 검사해 최대 3개 제한 적용
- 화면 연동: Signal/Slot 기반으로 등록·수정 결과를 조회 화면에 반영
- 정보 전달: 시간 정렬과 색상·완료 상태를 이용해 일정 식별 지원

03 성장과 학습

- 업무 요구사항을 UI 이벤트와 데이터 검증 규칙으로 구체화하는 경험
- 팀 공통 데이터와 본인 담당 화면을 연결하는 협업 경험

04 결과와 확인 범위

- 구현 결과: 예약 등록·조회·수정·삭제 및 예약 상한 검사
- 기능 연동: 담당 등록·조회 화면을 팀 공통 데이터 구조와 연결

08 카메라 기반 AI 모빌리티

차선 추적·객체 인식·모터 제어 통합 · 졸업 프로젝트

2023.09–2024.05 | 팀장 · 영상 처리 및 주행 시스템 개발 참여

카메라 입력부터 인식·판단·모터 제어까지 연결해 모의 트랙에서 동작하는 시스템 구현

Python / Raspberry Pi 4B / OpenCV / Keras / Bluetooth serial

PROCESS FLOW



팀 구현: 차선 추적과 사람 감지 시 정지·재출발 동작

01 본인 기여

- 프로젝트 운영: 팀장으로 졸업 프로젝트 진행 및 통합 개발 참여
- 인식·제어: OpenCV 영상 처리와 학습 모델을 활용한 주행 시스템 개발 참여

02 설계 판단과 문제 해결

- 입력 편향: 팀에서 좌·직진·우 주행 데이터를 균형 있게 수집하여 학습 입력 구성
- 판단 연결: 차선 추적 결과와 사람 감지 결과를 모터 명령에 연결
- 통합 확인: 모의 트랙에서 주행 및 정지 후 재출발 동작 확인

03 성장과 학습

- 센서 입력·추론·통신·구동기의 전체 처리 흐름을 구성한 경험
- 개별 기능 구현을 실제 하드웨어 동작으로 연결하는 통합 관점 습득

04 결과와 확인 범위

- 팀 결과: 트랙 내 차선 주행과 사람 감지 시 정지·재출발 확인
- 수상: 강원대학교 졸업작품 장려상, 2024.11.19 · 검증 범위는 모의 트랙

경험의 기반

경력 · 교육

IPG Automotive Korea

2025.12 - 현재

- Application팀 연구원 (HIL / ADAS)
- C 제어 로직·실시간 신호 매핑·HW/SW 통합 검증
- 시뮬레이션 플랫폼 업그레이드 및 현장 기술지원

Vision's Edge Device Academy

2025.03 - 2025.08

- 임베디드 Linux·디바이스 드라이버·ARM·Buildroot·Yocto 교육 이수
- 한화비전 최종 프로젝트 최우수상(1위) 및 우수 훈련생상

강원대학교 전기전자공학과

2018.03 - 2025.02

- 졸업 · 학점 3.55 / 4.5 · 전공 3.64 / 4.5
- 학부연구생 (2024.03 - 12): 센서 퓨전 연구용 데이터 처리

자격 · 어학

- 정보처리기사 · 2026.09.11 합격
- ISTQB CTFL · 2026.06 취득
- TOEIC Speaking IH · 2025.09 / TOEIC 820 · 2025.01

LET'S CONNECT

wldh0026@gmail.com

github.com/gitgio99

수상 기록

기술 구현 · 협업 · 학습 성과

2025.08.07 / 한화비전

최우수상 · 최종 최우수팀(1위)

이면도로 사각지대 사고방지 시스템

임베디드/비전 AI 교육과정 최종 프로젝트에서 기술력과 협업 능력을 인정받아 최우수팀 선정

2025.08.07 / 한화비전

우수 훈련생상

임베디드/비전 AI 교육과정

교육과정 전반의 학업 성적 및 실습 평가 우수자로 선정되어 표창 수여

2024.12.23 / (주)선도소프트

우수상

천안시 불법주정차 데이터 분석

공공데이터를 활용한 불법주정차 분석 아이디어의 활용성과 문제 해결 능력을 인정받아 수상

2024.11.19 / 강원대학교

장려상

차선 및 물체인식 AI모빌리티

학부 졸업작품의 전공 지식 적용과 작품 완성도를 인정받아 수상